Shell Sort e Selection Sort

Funcionamento, implementação e análise de desempenho



Iniciar

História do Shell Sort

TRÊS NOMES PARA LEMBRAR

1959

Donald L. Shell

Publicou o Shell Sort como uma melhoria da ordenação por inserção.

1973

Donald E. Knuth

Apresentou a sequência de gaps gerada por 3h + 1.

2001

Marcin Ciura

Publicou um estudo experimental de sequências para melhorar o desempenho médio.

Shell Sort aproxima os valores antes da última passagem.

Como funciona o Shell Sort

ORDENA PRIMEIRO OS VALORES MAIS DISTANTES

1 **Separar em grupos**

O gap define a distância entre as posições de cada grupo.

2 **Ordenar cada grupo**

Como na inserção: desloca os maiores e encaixa o valor.

3 **Diminuir o gap e repetir**

Com gap 1, todos ficam no mesmo grupo e a ordenação termina.

Os primeiros gaps aproximam os valores das posições finais.

Gaps e sua importância

O GAP É A DISTÂNCIA ENTRE AS POSIÇÕES

A escolha muda as passagens, os deslocamentos e o tempo.

Metades

Começa em $\lfloor n/2 \rfloor$ e divide por dois, com divisão inteira, até 1.

Para n = 100: 50, 25, 12, 6, 3, 1.

Knuth

1, 4, 13, 40, 121, 364, ...

Próximo gap = 3h + 1.

Ciura fixo

1, 4, 10, 23, 57, 132, 301, 701, 1750

Não usamos a versão estendida.

Usamos do maior para o menor, terminando em gap 1.

Exemplo: passagem com gap 4

OITO ELEMENTOS • GAPS POR METADES: 4, 2 E 1



Grupos de posições: (0, 4), (1, 5), (2, 6) e (3, 7).

No primeiro grupo, 8 e 6 viram 6 e 8. Os outros seguem a mesma ideia.

Mesma cor = mesmo grupo. Os números abaixo indicam as posições.

Exemplo: passagem com gap 2

OITO ELEMENTOS • GAPS POR METADES: 4, 2 E 1



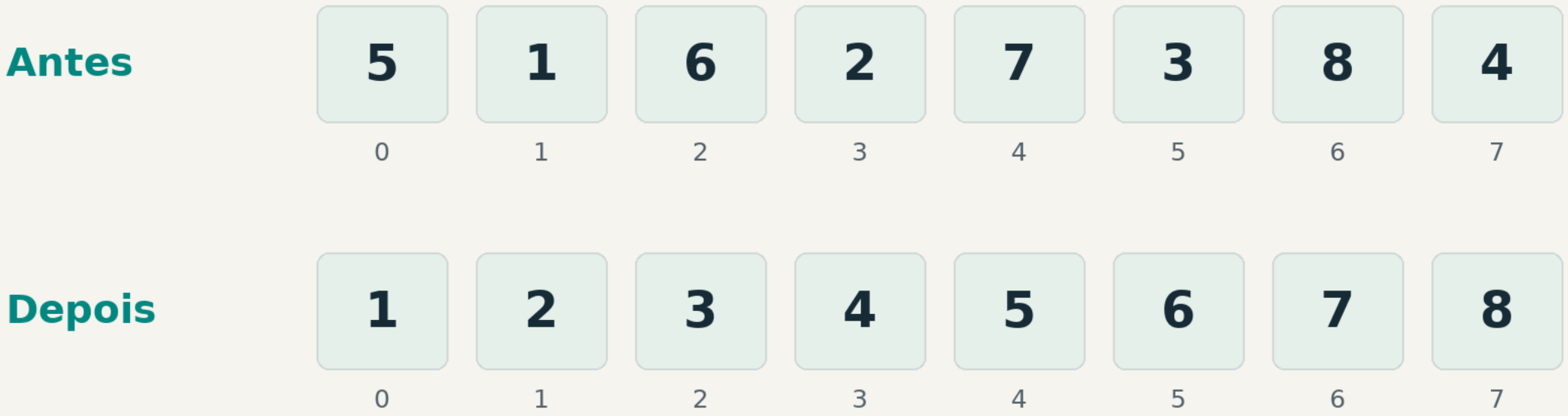
Agora os grupos são as posições (0, 2, 4, 6) e (1, 3, 5, 7).

Os grupos [6, 5, 8, 7] e [2, 1, 3, 4] ficam ordenados separadamente.

Mesma cor = mesmo grupo. Os números abaixo indicam as posições.

Exemplo: passagem com gap 1

OITO ELEMENTOS • GAPS POR METADES: 4, 2 E 1



Com gap 1, todas as posições formam um único grupo.

A inserção encaixa cada valor entre os anteriores. O vetor fica ordenado.

Mesma cor = mesmo grupo. Os números abaixo indicam as posições.

Shell Sort é estável?

NÃO GARANTE A ORDEM ORIGINAL DOS VALORES IGUAIS

2A e 2B têm o mesmo valor. As letras identificam cada elemento.

Entrada	**2A**	**2B**	**1**	**3**
Após gap 2	**1**	**2B**	**2A**	**3**
Após gap 1	**1**	**2B**	**2A**	**3**

O salto do gap 2 colocou 2A depois de 2B.

Mesmo com a inserção normal no final, essa ordem pode permanecer.

Complexidades das implementações

MELHOR E PIOR CASO DE CADA VERSÃO

Versão	Melhor caso	Pior caso
Metades	$\Theta(n \log n)$	$\Theta(n^2)$
Knuth	$\Theta(n \log n)$	$\Theta(n^{3/2})$
Ciura fixo até 1750	$\Theta(n)$	$\Theta(n^2)$

Ciura: o melhor caso linear vale para a nossa lista fixa, com no máximo nove passagens.

Outras sequências

REFERÊNCIAS DO RELATÓRIO: NÃO IMPLEMENTADAS NEM TESTADAS

Hibbard

$2^k - 1$: 1, 3, 7, 15, 31, ...

$O(n^{3/2})$

Pratt

$2^p 3^q$: 1, 2, 3, 4, 6, 8, 9, 12, ...

$O(n \log^2 n)$

Sedgewick

Sequência específica: 1, 8, 23, 77, 281, ...

$O(n^{4/3})$

Limites de pior caso. A escolha dos gaps pode alterar a complexidade.

Shell: vantagens e desvantagens

POUCA MEMÓRIA EXTRA, MAS O DESEMPENHO DEPENDE DOS GAPS

Vantagens

Memória auxiliar constante.

Move valores distantes
nas primeiras passagens.

Permite escolher diferentes
sequências de gaps.

Desvantagens

Não é estável.

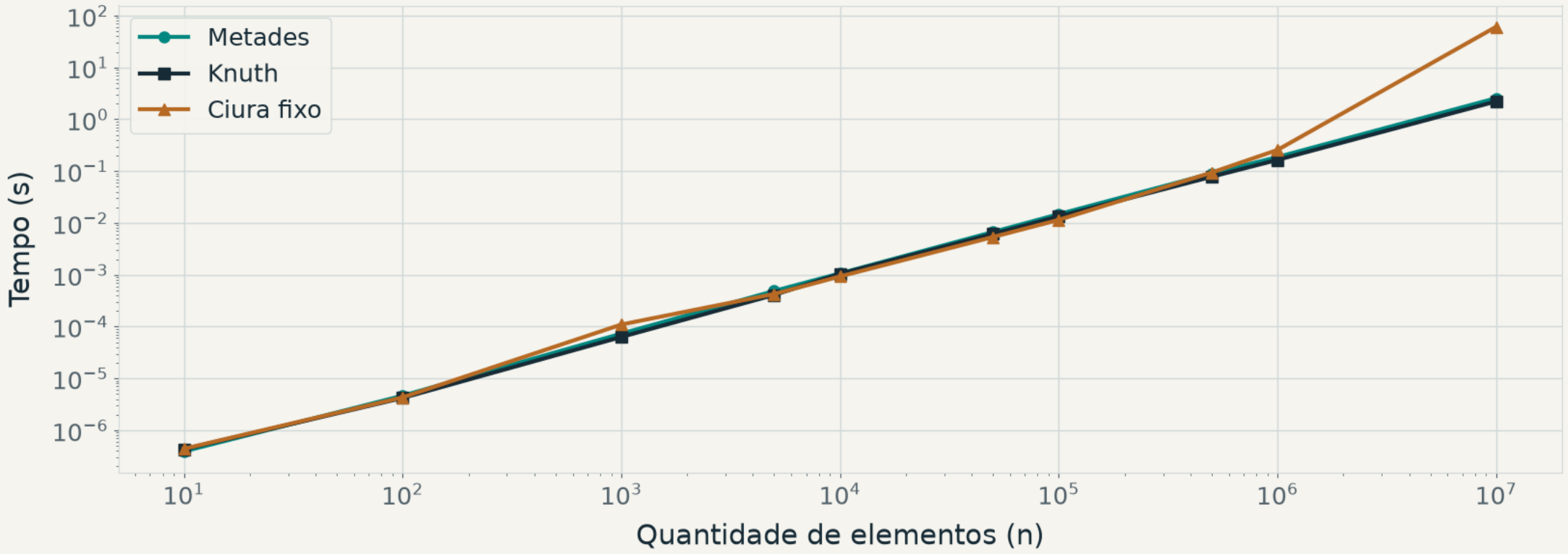
O desempenho depende
da sequência de gaps.

Algumas sequências têm
pior caso quadrático.

A análise do pior caso é mais difícil que a do Selection Sort.

Tempo — entrada aleatória

SHELL SORT • METADES, KNUTH E CIURA FIXO



Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Tabela: tempo — entrada aleatória

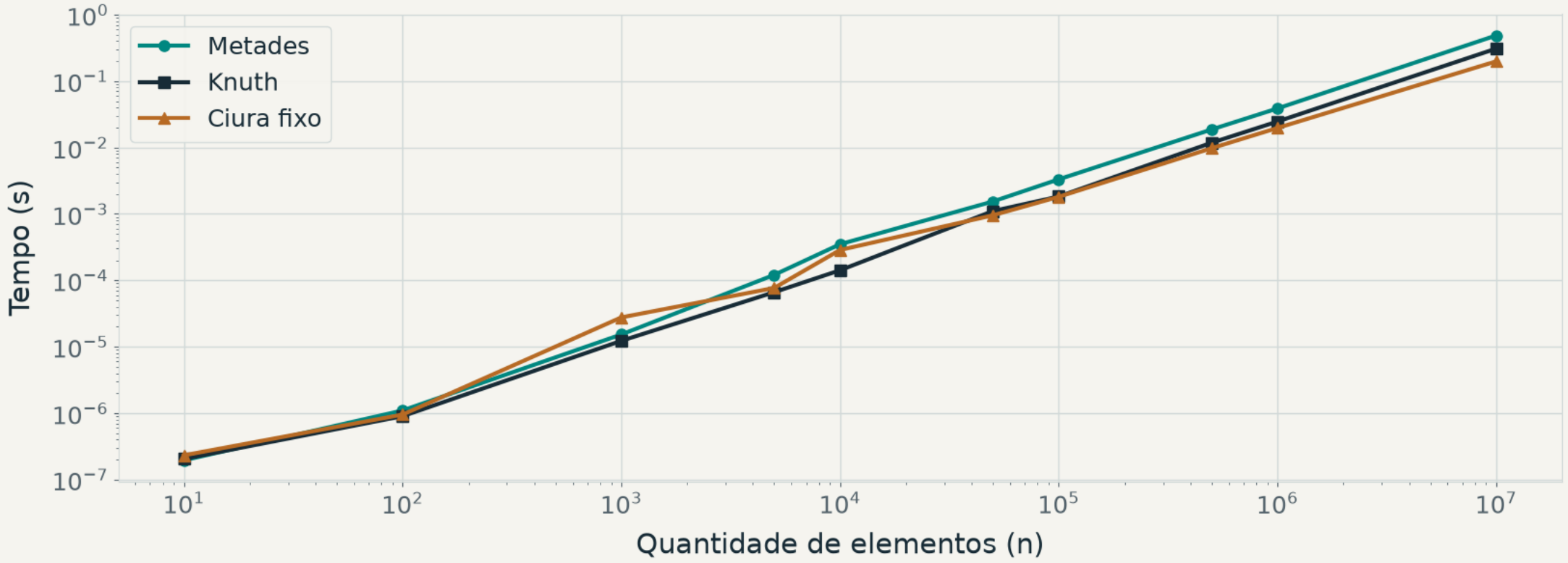
SHELL SORT • TEMPO (S)

n	Metades	Knuth	Ciura fixo
10	0,000000390	0,000000429	0,000000438
100	0,000004631	0,000004194	0,000004305
1.000	0,000072856	0,000063018	0,000109526
5.000	0,000486612	0,000408208	0,000425297
10.000	0,001062181	0,001042302	0,000928731
50.000	0,006674334	0,006113780	0,005347633
100.000	0,014752876	0,013344030	0,011474615
500.000	0,090441741	0,077902669	0,093830671
1.000.000	0,186632804	0,164819346	0,256181891
10.000.000	2,559364680	2,219851804	61,100016282

Uma medição por combinação. Tempos e contagens não são médias.

Tempo — entrada crescente

SHELL SORT • METADES, KNUTH E CIURA FIXO



Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Tabela: tempo — entrada crescente

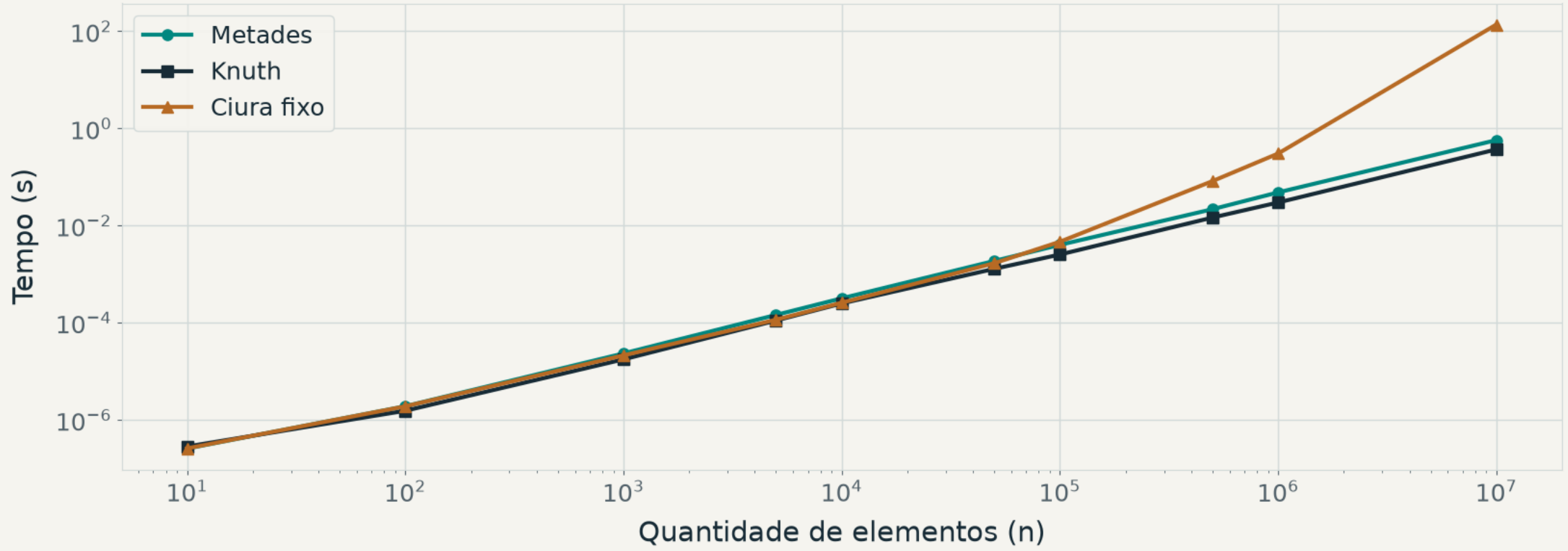
SHELL SORT • TEMPO (S)

n	Metades	Knuth	Ciura fixo
10	0,000000195	0,000000205	0,000000232
100	0,000001099	0,000000901	0,000000953
1.000	0,000015308	0,000012286	0,000027688
5.000	0,000120905	0,000066347	0,000076838
10.000	0,000349752	0,000141883	0,000286576
50.000	0,001540631	0,001104427	0,000954213
100.000	0,003321665	0,001830074	0,001811387
500.000	0,018699776	0,011814468	0,009763198
1.000.000	0,038767026	0,024633379	0,019712788
10.000.000	0,486784363	0,308631088	0,198378865

Uma medição por combinação. Tempos e contagens não são médias.

Tempo — entrada decrescente

SHELL SORT • METADES, KNUTH E CIURA FIXO



Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Entrada decrescente não é, automaticamente, o pior caso matemático.

Tabela: tempo — entrada decrescente

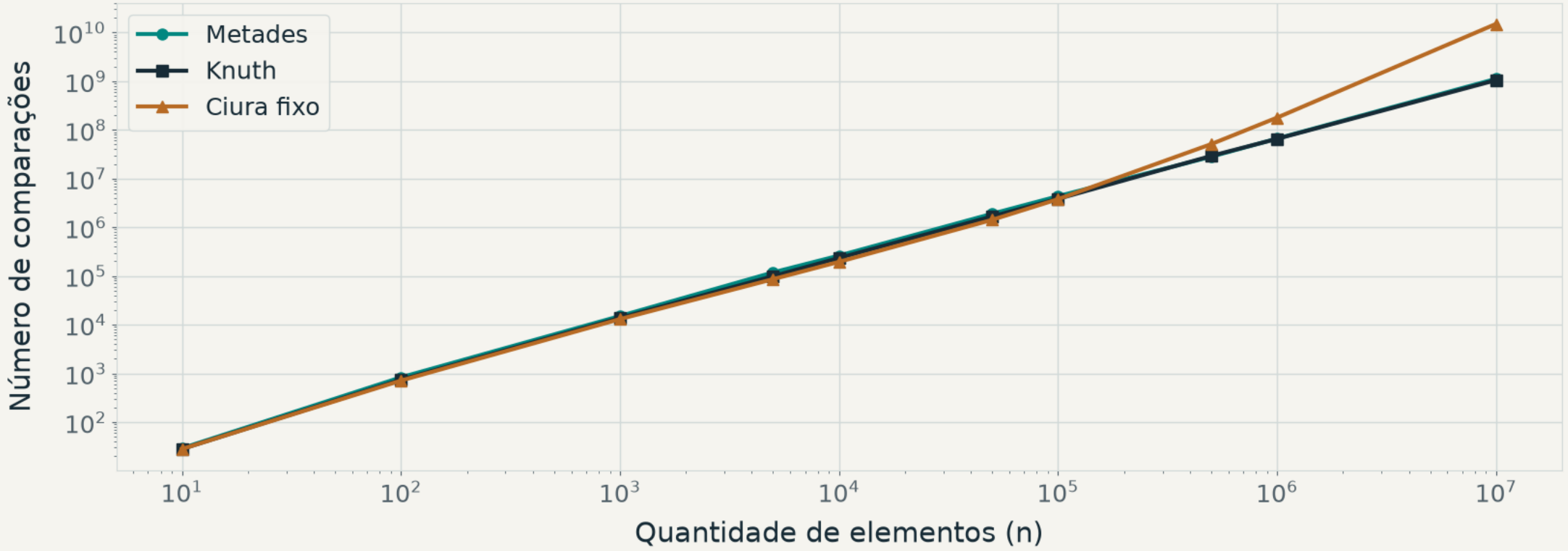
SHELL SORT • TEMPO (S)

n	Metades	Knuth	Ciura fixo
10	0,000000255	0,000000281	0,000000257
100	0,000001894	0,000001529	0,000001888
1.000	0,000023160	0,000017477	0,000021130
5.000	0,000143682	0,000110063	0,000114380
10.000	0,000315406	0,000247542	0,000256048
50.000	0,001854750	0,001274804	0,001665466
100.000	0,003970621	0,002514812	0,004609766
500.000	0,021537431	0,014497618	0,081530552
1.000.000	0,047809143	0,029639088	0,301458130
10.000.000	0,570087394	0,366575925	136,216577672

Uma medição por combinação. Tempos e contagens não são médias.

Comparações — entrada aleatória

SHELL SORT • METADES, KNUTH E CIURA FIXO



Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Tabela: comparações — entrada aleatória

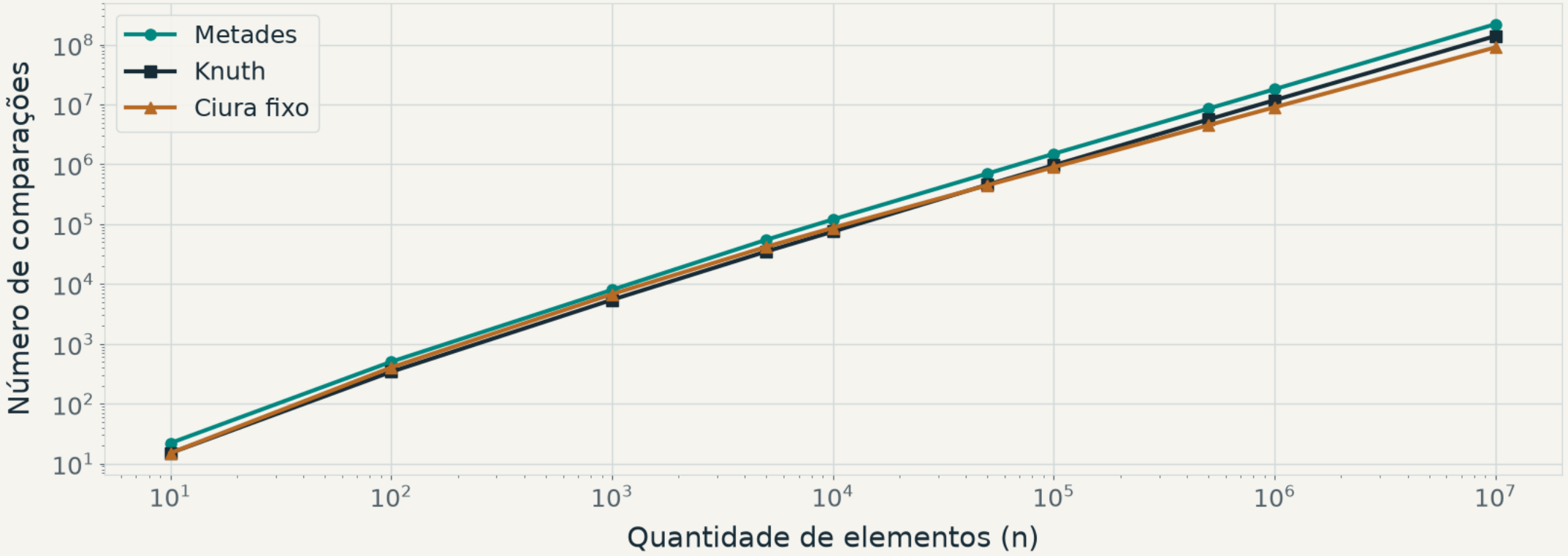
SHELL SORT • NÚMERO DE COMPARAÇÕES

n	Metades	Knuth	Ciura fixo
10	29	28	28
100	829	739	711
1.000	15.035	13.739	13.014
5.000	118.125	98.789	86.009
10.000	261.534	233.623	194.969
50.000	1.888.996	1.666.757	1.419.087
100.000	4.338.950	3.830.202	3.772.164
500.000	28.072.970	29.080.324	50.791.358
1.000.000	66.121.589	65.318.817	177.825.638
10.000.000	1.113.901.611	1.047.831.989	14.949.158.849

Uma medição por combinação. Tempos e contagens não são médias.

Comparações — entrada crescente

SHELL SORT • METADES, KNUTH E CIURA FIXO



Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Tabela: comparações — entrada crescente

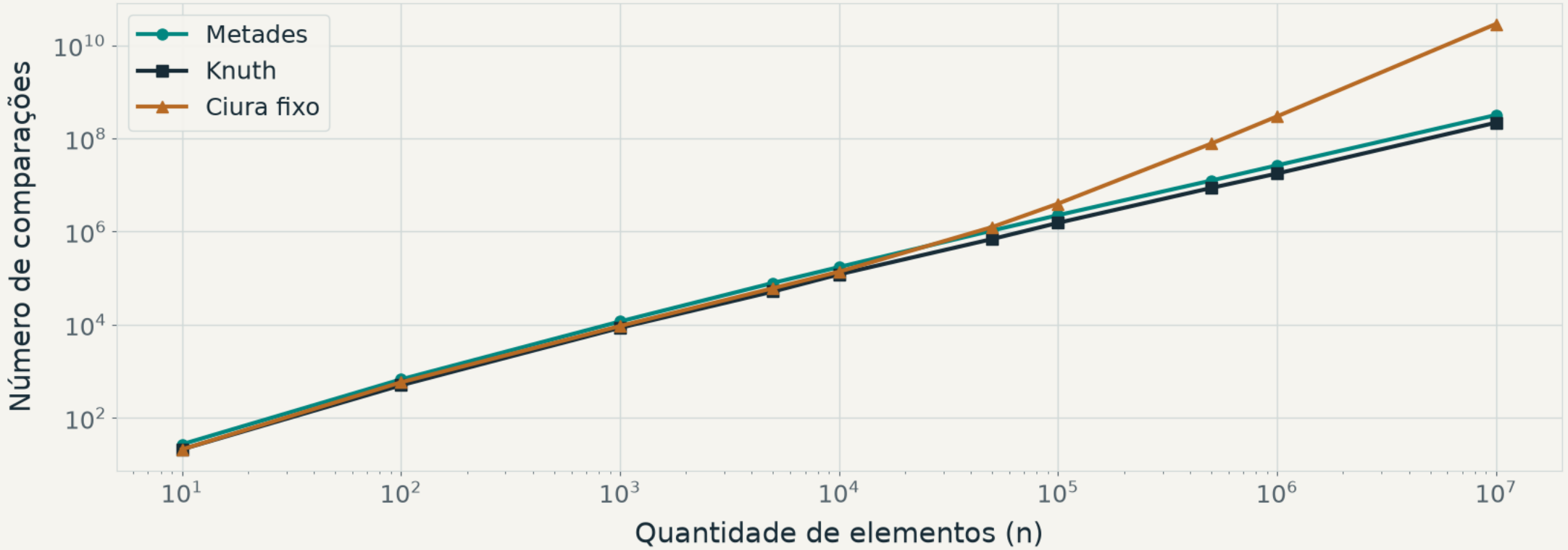
SHELL SORT • NÚMERO DE COMPARAÇÕES

n	Metades	Knuth	Ciura fixo
10	22	15	15
100	503	342	405
1.000	8.006	5.457	6.771
5.000	55.005	35.084	42.021
10.000	120.005	75.243	87.021
50.000	700.006	455.719	447.021
100.000	1.500.006	967.146	897.021
500.000	8.500.007	5.601.426	4.497.021
1.000.000	18.000.007	11.804.265	8.997.021
10.000.000	220.000.008	139.238.328	89.997.021

Uma medição por combinação. Tempos e contagens não são médias.

Comparações — entrada decrescente

SHELL SORT • METADES, KNUTH E CIURA FIXO



Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Entrada decrescente não é, automaticamente, o pior caso matemático.

Tabela: comparações — entrada decrescente

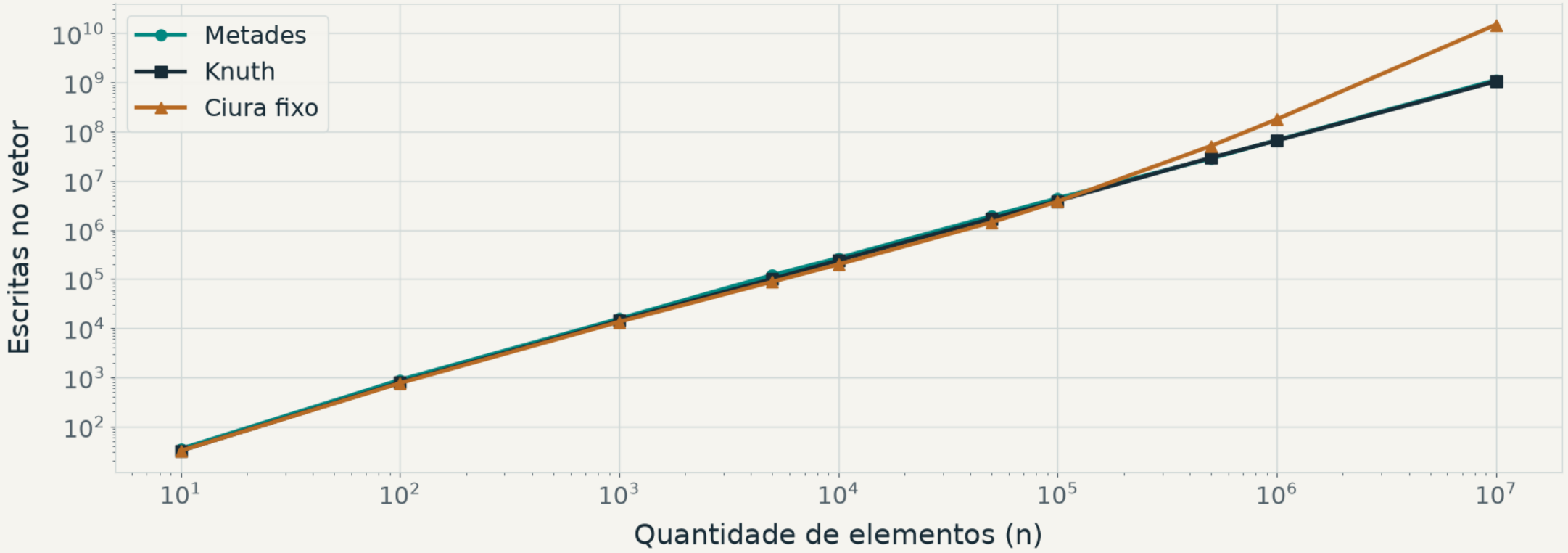
SHELL SORT • NÚMERO DE COMPARAÇÕES

n	Metades	Knuth	Ciura fixo
10	27	21	21
100	668	500	579
1.000	11.716	8.550	9.354
5.000	78.798	51.491	60.712
10.000	172.578	120.190	137.075
50.000	1.047.305	689.218	1.246.058
100.000	2.244.585	1.533.494	3.968.350
500.000	12.428.778	8.622.570	76.944.995
1.000.000	26.357.530	17.578.626	296.380.356
10.000.000	317.626.219	217.670.525	28.681.792.543

Uma medição por combinação. Tempos e contagens não são médias.

Movimentações — entrada aleatória

SHELL SORT • METADES, KNUTH E CIURA FIXO



Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Escritas no vetor, incluindo a gravação final do temporário.

Tabela: movimentações — entrada aleatória

SHELL SORT • ESCRITAS NO VETOR

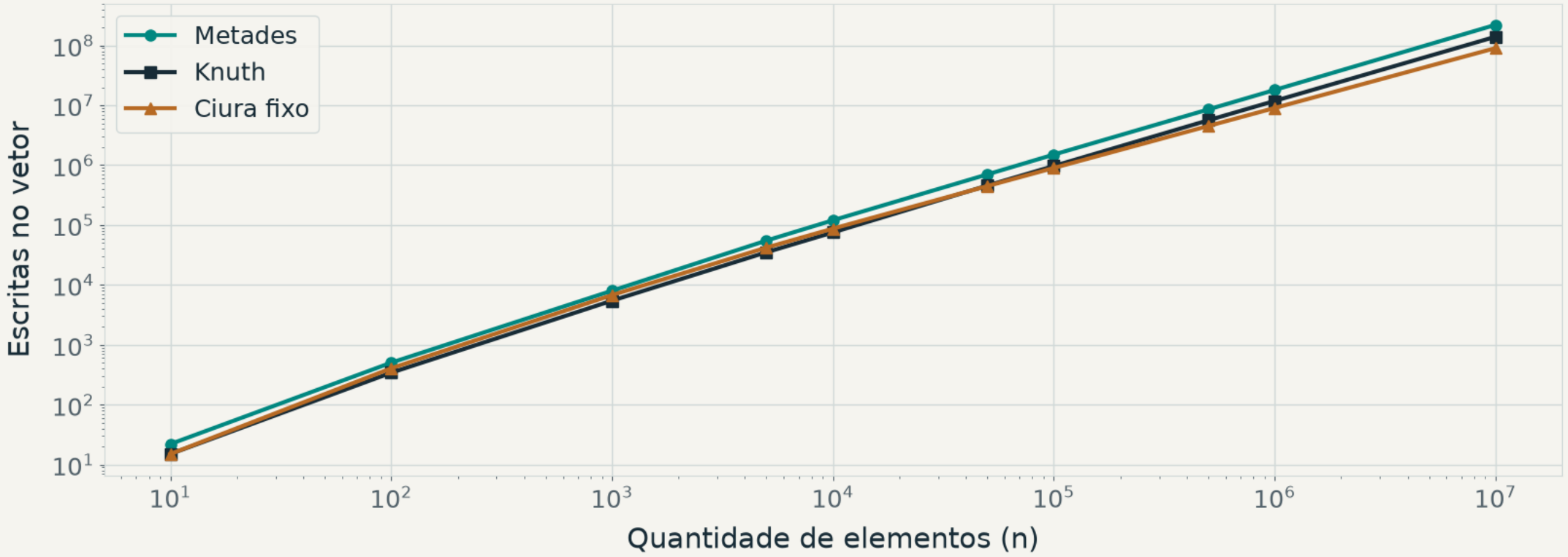
n	Metades	Knuth	Ciura fixo
10	35	32	32
100	885	784	761
1.000	15.547	14.174	13.462
5.000	120.673	100.966	88.255
10.000	266.629	237.985	198.441
50.000	1.914.433	1.689.682	1.425.324
100.000	4.389.591	3.871.711	3.779.446
500.000	28.325.796	29.317.469	50.801.744
1.000.000	66.623.940	65.727.764	177.837.084
10.000.000	1.118.919.386	1.051.977.554	14.949.174.355

Uma medição por combinação. Tempos e contagens não são médias.

Movimentações incluem a gravação final do temporário.

Movimentações — entrada crescente

SHELL SORT • METADES, KNUTH E CIURA FIXO



Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Escritas no vetor, incluindo a gravação final do temporário.

Tabela: movimentações — entrada crescente

SHELL SORT • ESCRITAS NO VETOR

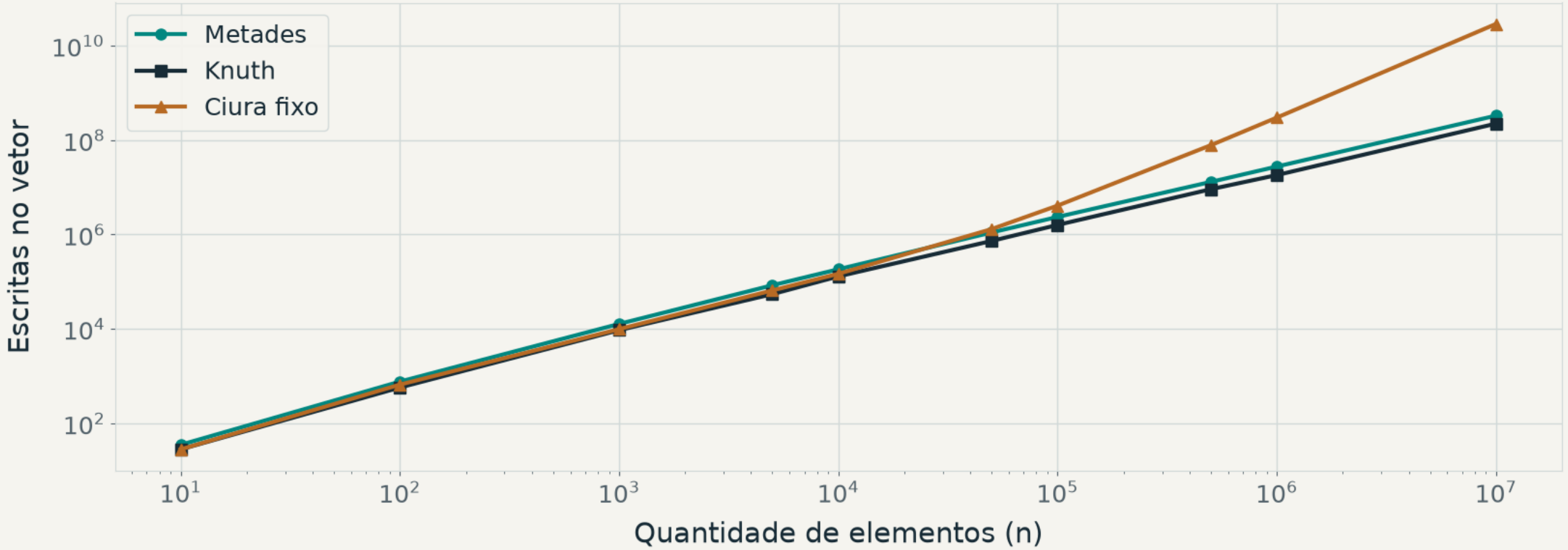
n	Metades	Knuth	Ciura fixo
10	22	15	15
100	503	342	405
1.000	8.006	5.457	6.771
5.000	55.005	35.084	42.021
10.000	120.005	75.243	87.021
50.000	700.006	455.719	447.021
100.000	1.500.006	967.146	897.021
500.000	8.500.007	5.601.426	4.497.021
1.000.000	18.000.007	11.804.265	8.997.021
10.000.000	220.000.008	139.238.328	89.997.021

Uma medição por combinação. Tempos e contagens não são médias.

Movimentações incluem a gravação final do temporário.

Movimentações — entrada decrescente

SHELL SORT • METADES, KNUTH E CIURA FIXO



Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Escritas no vetor, incluindo a gravação final do temporário.

Tabela: movimentações — entrada decrescente

SHELL SORT • ESCRITAS NO VETOR

n	Metades	Knuth	Ciura fixo
10	35	28	28
100	763	572	659
1.000	12.706	9.377	9.861
5.000	83.785	54.296	65.105
10.000	182.565	128.947	146.331
50.000	1.097.286	726.487	1.294.815
100.000	2.344.566	1.586.800	4.067.357
500.000	12.928.759	9.053.230	77.444.251
1.000.000	27.357.511	18.049.649	297.379.027
10.000.000	327.626.184	221.971.074	28.691.791.291

Uma medição por combinação. Tempos e contagens não são médias.

Movimentações incluem a gravação final do temporário.

Conclusão: a escolha dos gaps

NENHUMA SEQUÊNCIA VENCEU TODOS OS CENÁRIOS

Knuth

Foi o mais rápido no teste decrescente com 10 milhões de elementos.

Ciura fixo

Foi mais rápido nas entradas crescentes maiores, mas teve alto custo na decrescente. Sua lista termina em 1750.

Tamanho e ordem da entrada

Os resultados mudaram conforme o tamanho do vetor e sua organização.

Uma medição por teste: repetir ajudaria a avaliar a variação dos tempos.

Os resultados medidos não são uma garantia para qualquer entrada.

Selection Sort

História, código em C e análise de desempenho



Projeto e Análise de Algoritmos | 2026

Iniciar

Como surgiu o Selection Sort?

A ideia é simples

Procurar o menor valor, colocá-lo no início e repetir com os restantes.

Nos anos 1950

Pesquisadores já estudavam formas de ordenar dados nos primeiros computadores.

É dessa época que vêm alguns dos registros mais antigos de métodos de seleção.

Base histórica: Rochester (2021) e estudo de Owen Astrachan, Duke (2003).

1956: um dos primeiros registros

1956

Howard B. Demuth

Estudou formas de ordenar dados
na Universidade de Stanford.

Em seu doutorado, ele descreveu uma técnica que depois foi relacionada ao Selection Sort.

É um registro antigo, mas não prova quem inventou o algoritmo.

Fonte principal: Alan Beadle, Historical Origins of Data Structures and Algorithms (2021), p. 24.

Outros estudos em 1956 e 1958

1956

Edward H. Friend

Publicou um estudo sobre como ordenar dados em computadores.

1958

D. A. Bell

Também apresentou métodos para ordenar informações.

Esses métodos eram parecidos, mas não exatamente iguais à versão atual.

Análise histórica: Owen Astrachan (Duke, 2003), seção 2.2.

Como passou a ser ensinado

1973 Knuth publica um livro importante sobre algoritmos de ordenação.

1998 Uma edição do livro traz um capítulo sobre ordenação por seleção.

2000 O MIT apresenta Selection Sort em suas aulas de programação.

Fontes: bibliografia de Knuth, editora Addison-Wesley / InformIT e MIT OpenCourseWare.

O que lembrar sobre a história?

Época	O que sabemos
Anos 1950	Já existiam estudos de métodos de seleção.
1956 e 1958	Pesquisadores publicaram trabalhos de ordenação.
Anos 1970–1990	O assunto aparece em livros de algoritmos.
Ano 2000	Selection Sort já era ensinado em cursos, como no MIT.

Não há um inventor comprovado do Selection Sort.

Shell Sort, publicado por Donald Shell em 1959, é outro algoritmo.

Como funciona

A cada rodada, o algoritmo encontra o menor valor da parte restante e o troca com a primeira posição livre.



O processo continua até todas as posições ficarem definidas.

Ver exemplo

Exemplo: qual é o menor valor?

Clique no menor para revelar a resposta. Outros iniciam as comparações.

64	25	12	22	11
índice 0	índice 1	índice 2	índice 3	índice 4

Objetivo: colocar os cinco valores em ordem crescente.

Interação disponível no modo Apresentação de Slides (F5).

Comparar passo a passo

1ª rodada: comparação com 25

25 < 64

25 é menor. Agora ele é o candidato.



Menor até agora Em comparação Posição definida

Durante a busca, os valores permanecem nas mesmas posições.

Voltar Próxima comparação

1ª rodada: comparação com 12

12 < 25

12 é menor. O candidato passa a ser 12.



Menor até agora Em comparação Posição definida

Durante a busca, os valores permanecem nas mesmas posições.

Voltar Próxima comparação

1ª rodada: comparação com 22

22 > 12

22 é maior. O candidato continua sendo 12.



Menor até agora Em comparação Posição definida

Durante a busca, os valores permanecem nas mesmas posições.

Voltar Próxima comparação

1ª rodada: comparação com 11

11 < 12

11 é o menor de todos. A busca terminou.



Menor até agora Em comparação Posição definida

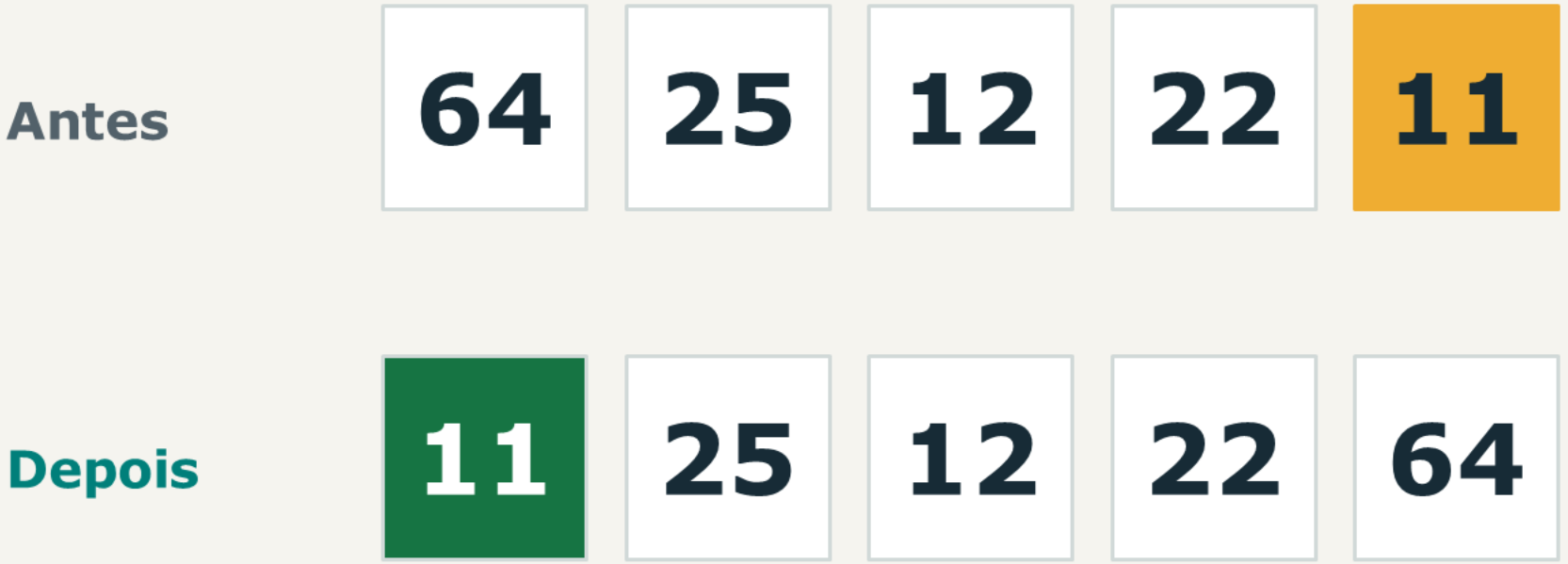
Durante a busca, os valores permanecem nas mesmas posições.

Voltar

Realizar troca

A troca: 11 ocupa a primeira posição

Após encontrar o menor, o algoritmo troca 64 e 11.



11 está na posição definitiva. A próxima busca começa no 25.

Voltar

Continuar rodadas

As próximas rodadas

O trecho já ordenado cresce uma posição por rodada.

2ª rodada



Menor restante: 12. Troca com 25.

3ª rodada



Menor restante: 22. Troca com 25.

4ª rodada



25 já está no lugar. 64 fica por último.

Resultado: 11, 12, 22, 25, 64

Recomeçar | Síntese

A ideia central do Selection Sort

Selecionar o menor restante e colocá-lo na próxima posição.

Busca

Compara os valores e guarda a posição do menor.

Troca

Move o menor para o início da parte ainda não ordenada.

A parte ordenada permanece pronta nas rodadas seguintes.

Rever exemplo

Código em C

Implementação real do trabalho

O PROGRAMA USA ARQUIVOS DE ENTRADA, NÃO UM VETOR DIGITADO

1 **Ler dados**

Arquivos .txt com n inteiros.

2 **Ordenar**

selection_sort() em C.

3 **Registrar**

Comparações, movimentos e tempo.

30 execuções = 10 tamanhos × 3 tipos de entrada

Aleatória • Crescente • Decrescente

Os arquivos de teste devem existir antes de iniciar a bateria; este código apenas os lê.

Código real

Código em C: Selection Sort

TRECHO RETIRADO DA FUNÇÃO selection_sort() ENVIADA

```
static void selection_sort(int *v, size_t n) {
    for (size_t i = 0; i + 1 < n; ++i) {
        size_t menor = i;
        for (size_t j = i + 1; j < n; ++j) {
            comparacoes++;
            if (v[j] < v[menor]) menor = j;
        }
        if (menor != i) {
            int tmp = v[i];
            v[i] = v[menor];
            v[menor] = tmp;
            movimentacoes += 2;
        }
    }
}
```

i e j

i define a posição; j percorre o restante.

comparacoes++

Soma 1 ao comparar valores.

movimentacoes += 2

Troca efetiva = 2 escritas no vetor.

Como o programa executa cada teste

FUNÇÕES PRESENTES NO ARQUIVO selection_sort(1).c

main()	Localiza a pasta, verifica os 30 arquivos e coordena os testes.
carregar()	Lê exatamente n inteiros do arquivo .txt.
executar()	Aloca memória, ordena, valida e preenche Resultado.
segundos()	Calcula a diferença entre início e fim em segundos.
iniciar_relatorios()	Cria arquivos selection.csv e selection.txt.
finalizar_relatorios()	Registra a duração total do programa e fecha os relatórios.

O vetor é liberado com free() após cada execução, inclusive nos caminhos de erro.

Bateria de testes: 30 combinações

10 TAMANHOS DE VETOR E TRÊS ORGANIZAÇÕES DE ENTRADA

ALEATÓRIO	CRESCENTE	DECRESCENTE
Números em ordem não organizada	Vetor já em ordem	Valores inicialmente invertidos

Tamanhos utilizados (n)

10	**100**	**1.000**	**5.000**	**10.000**
50.000	**100.000**	**500.000**	**1.000.000**	**10.000.000**

1 medição por combinação (não são médias). O código exige os 30 arquivos previamente.

O que é contabilizado?

COMPARAÇÕES, TROCAS E MOVIMENTAÇÕES NÃO SÃO A MESMA COISA

Comparações

comparacoes++ é executado no laço interno, independentemente de ocorrer troca.

Movimentações

Uma troca entre posições diferentes soma exatamente 2 escritas no vetor.

Exemplo com 5 elementos

[64, 25, 12, 22, 11] **10 comparações** **3 trocas = 6 movimentações**

Se o vetor já estiver ordenado: 10 comparações, 0 trocas e 0 movimentações.

Melhor caso e pior caso: $\Theta(n^2)$

A CONTAGEM DE COMPARAÇÕES NÃO DEPENDE DA ORDEM DOS VALORES

$$C(n) = (n - 1) + (n - 2) + \cdots + 1 = n(n - 1) / 2$$

Melhor caso

Entrada já crescente: não ocorre troca. Mas o laço interno continua fazendo todas as comparações.

Pior caso

Mesmo com entrada desfavorável, a contagem de comparações continua sendo $n(n - 1)/2$.

$$O(n^2) + \Omega(n^2) \Rightarrow \Theta(n^2) \text{ em ambos os casos}$$

Crescimento quadrático na prática

AS COMPARAÇÕES CRESCEM MUITO MAIS RÁPIDO DO QUE n

n = 5	**10** comparações
n = 100	**4.950** comparações
n = 1.000	**499.500** comparações
n = 10.000	**49.995.000** comparações

Multiplicar n por 10 aumenta a contagem de comparações em aproximadamente 100 vezes.

Estabilidade e uso de memória

A VERSÃO COM TROCA DIRETA NÃO É ESTÁVEL

Entrada



Após a 1ª troca



A ordem de 2A e 2B foi invertida.

Ordenação: Θ(1) de memória extra • Até n − 1 trocas

O programa completo também reserva um vetor de entrada com malloc(): Θ(n) de armazenamento.

Vantagens

Vantagens e desvantagens

SIMPLES DE ENTENDER, MAS CUSTOSO PARA ENTRADAS GRANDES

Vantagens

- Código curto e fácil de explicar
- Memória auxiliar $\Theta(1)$
- No máximo $n - 1$ trocas efetivas

Desvantagens

- $\Theta(n^2)$ mesmo se a entrada já está em ordem
- Não preserva a ordem dos iguais
- Tempo elevado para vetores grandes

Menos trocas não garantem menor tempo: o programa continua procurando o menor.

Como o desempenho foi medido

RELATÓRIOS CSV/TXT E VALIDAÇÃO APÓS CADA ORDENAÇÃO

1 **Cronômetro**

clock_gettime(): relógio monotônico

2 **Trecho medido**

Somente selection_sort()

3 **Conferência**

Vetor final em ordem crescente

selection.csv **selection.txt**

Tipo de entrada, n, tempo, comparações, movimentações e validação.

Os resultados publicados são medições individuais, não médias de várias execuções.

Resultados

Resultados registrados no relatório

TESTE DE 10 MILHÕES DE ELEMENTOS ALEATÓRIOS

Selection Sort

107.310,582 s

aproximadamente 29 h e 49 min

Para contextualizar

Shell Sort por metades

2,559 s

Selection realiza 49.999.995.000.000 comparações com n = 10.000.000.

Valores informados no PDF do trabalho; não foram executados novamente nesta atualização.

Conclusões: parte Selection Sort

SÍNTESE DA IMPLEMENTAÇÃO E DA ANÁLISE

1

A quantidade de comparações é fixa

C(n) = n(n − 1) / 2, em qualquer ordem inicial.

2

Movimentações dependem do vetor

Uma troca efetiva gera duas escritas no vetor.

3

Em vetores grandes, o custo domina

$O(n^2)$ e $\Omega(n^2)$ resultam em $\Theta(n^2)$.

Início

Computador utilizado nos testes

DELL OPTIPLEX 3060

Processador	Intel Core i5-8500T, 8ª geração
Frequência base	2,10 GHz
Núcleos e threads	6 núcleos e 6 threads
Memória RAM	16 GB — 2 módulos de 8 GB
Tecnologia da RAM	DDR4, 2400 MT/s
Armazenamento	SSD NVMe ADATA SM2P41C3, 256 GB
Sistema operacional	Ubuntu Server

Configuração informada no relatório do trabalho.

Selection Sort × Shell Sort

COMPARAÇÃO COM SHELL POR METADES

Selection Sort

Melhor caso: $\Theta(n^2)$
Pior caso: $\Theta(n^2)$
Busca o menor a cada rodada.

Shell por metades

Melhor caso: $\Theta(n \log n)$
Pior caso: $\Theta(n^2)$
Aproveita a organização dos gaps.

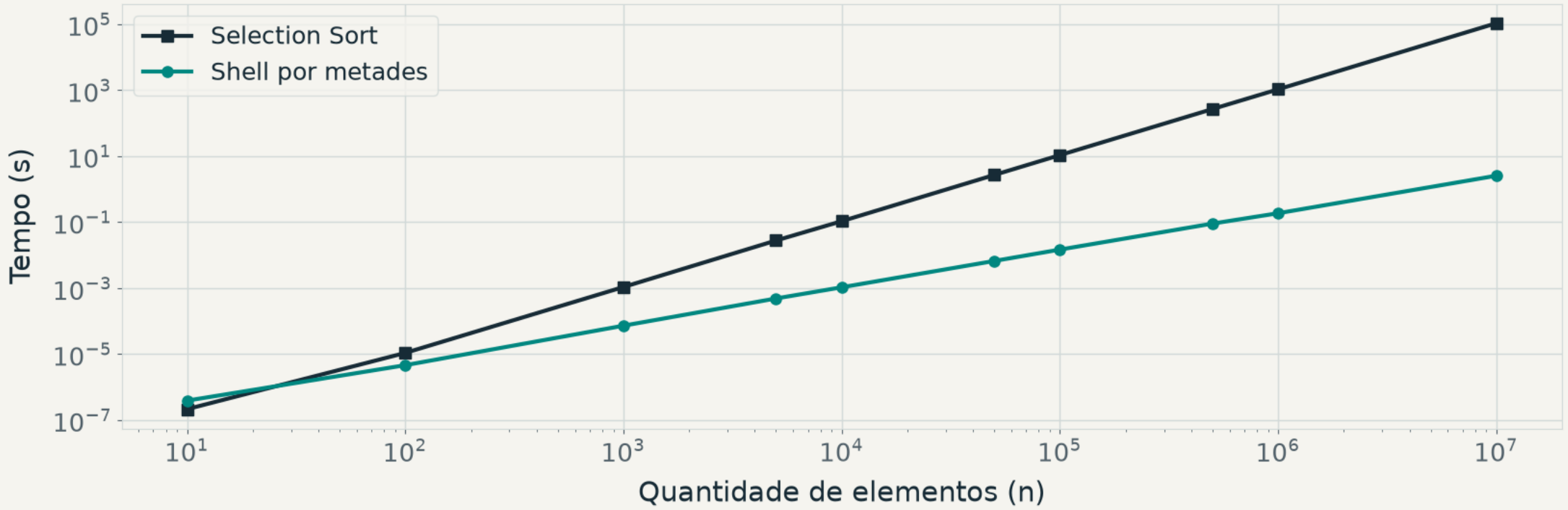
Ambos: memória auxiliar constante e sem garantia de estabilidade.

Mesmo pior caso não significa o mesmo tempo em todos os testes.

Nesta comparação, usamos somente Selection e Shell por metades.

Tempos — entrada aleatória

SELECTION SORT × SHELL POR METADES



10 milhões de elementos:

Selection: 107.310,582 s Shell por metades: 2,559 s

Escalas logarítmicas. Medições individuais do relatório, sem médias.

Tabela de tempos — entrada aleatória

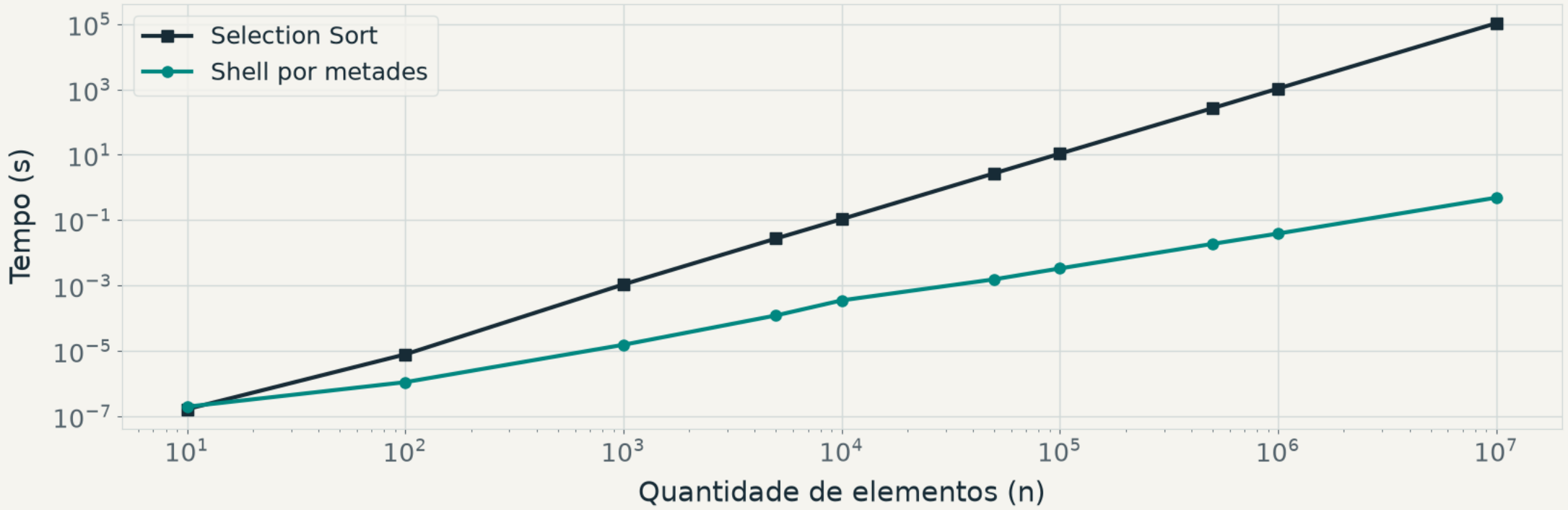
SELECTION × SHELL POR METADES • TEMPO EM SEGUNDOS

n	Selection Sort	Shell por metades
10	0,000000215	0,000000390
100	0,000010901	0,000004631
1.000	0,001083199	0,000072856
5.000	0,027786688	0,000486612
10.000	0,107050143	0,001062181
50.000	2,695467939	0,006674334
100.000	10,777727863	0,014752876
500.000	267,950135405	0,090441741
1.000.000	1062,470905078	0,186632804
10.000.000	107310,581887928	2,559364680

Uma medição por combinação. Tempos e contagens não são médias.

Tempos — entrada crescente

SELECTION SORT × SHELL POR METADES



10 milhões de elementos:

Selection: 105.912,840 s Shell por metades: 0,487 s

Escalas logarítmicas. Medições individuais do relatório, sem médias.

Tabela de tempos — entrada crescente

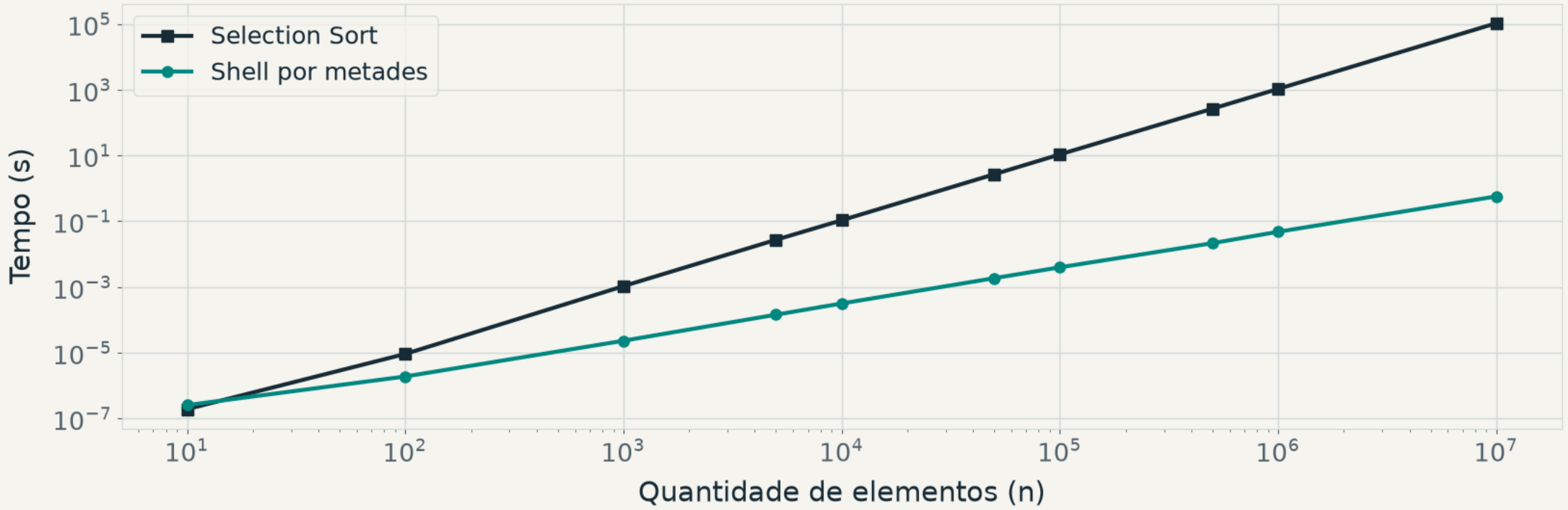
SELECTION × SHELL POR METADES • TEMPO EM SEGUNDOS

n	Selection Sort	Shell por metades
10	0,000000164	0,000000195
100	0,000007785	0,000001099
1.000	0,001075995	0,000015308
5.000	0,027275822	0,000120905
10.000	0,107220236	0,000349752
50.000	2,699136068	0,001540631
100.000	10,789414413	0,003321665
500.000	265,359122352	0,018699776
1.000.000	1061,405770046	0,038767026
10.000.000	105912,839734180	0,486784363

Uma medição por combinação. Tempos e contagens não são médias.

Tempos — entrada decrescente

SELECTION SORT × SHELL POR METADES



10 milhões de elementos:

Selection: 105.923,688 s Shell por metades: 0,570 s

Escalas logarítmicas. Medições individuais do relatório, sem médias.

Tabela de tempos — entrada decrescente

SELECTION × SHELL POR METADES • TEMPO EM SEGUNDOS

n	Selection Sort	Shell por metades
10	0,000000192	0,000000255
100	0,000009186	0,000001894
1.000	0,001062472	0,000023160
5.000	0,027273864	0,000143682
10.000	0,107271368	0,000315406
50.000	2,686454269	0,001854750
100.000	10,800248744	0,003970621
500.000	265,425003356	0,021537431
1.000.000	1061,469464228	0,047809143
10.000.000	105923,688492376	0,570087394

Uma medição por combinação. Tempos e contagens não são médias.

Menos escritas não garantem menor tempo

10 MILHÕES DE ELEMENTOS • ENTRADA ALEATÓRIA

Selection Sort

Comparações

49.999.995.000.000

Escritas no vetor

19.999.976

Shell por metades

Comparações

1.113.901.611

Escritas no vetor

1.118.919.386

Shell escreveu mais, mas fez muito menos comparações.

Na entrada crescente, Selection não fez trocas,
mas continuou fazendo todas as buscas.

Conclusão: Selection e Shell

O QUE OBSERVAMOS NOS TESTES

1 **Shell por metades foi mais rápido nos vetores grandes**

Com 10 milhões aleatórios: quase 30 horas contra cerca de 2,559 s.

2 **Selection não aproveitou a entrada já ordenada**

Sem trocas, mas com a mesma quantidade de comparações.

3 **O pior caso não descreve todas as entradas**

Ambos têm pior caso quadrático, mas os tempos medidos foram diferentes.

Precisamos olhar tempo, comparações e escritas em conjunto.

Bibliografia do Shell Sort

REFERÊNCIAS

Donald E. Knuth

The Art of Computer Programming, volume 3: Sorting and Searching.

Marcin Ciura (2001)

Estudo experimental de sequências de gaps.

Robert Sedgewick (1986 e 1996)

Limites e referências para outras sequências.

Abrir também o estudo de 1996

Resultados: shell_metades, shell_knuth e shell_ciura (CSV).

Uma medição por combinação. Sem execução nova nem médias.

Comparação e conclusão: relatório do trabalho, seções 4 e 7.